Statement of Qualifications

Uriah Raffer | JC-524404 | Security Operations Associate

Describe your experience supporting Security Operations activities, including monitoring security tools, analyzing alerts or logs, performing initial triage of security incidents, and assisting with basic forensic data collection under supervision. Include specific tools or methods you used, your role in incident handling, and how your work contributed to improving security processes.

My security operations experience comes from my cybersecurity internship at LOGN Pacific, where I work inside a Cyber Range built on actual enterprise infrastructure from Microsoft Azure and Tenable. I ramped up through the internship's self-paced classroom tracks one discipline at a time, with hands-on Security Information and Event Management (SIEM) training in Microsoft Sentinel and vulnerability management labs on the Tenable platform, and I applied each tool as I learned it. I've monitored Windows 10/11 endpoints and Linux hosts with Microsoft Defender for Endpoint.

Within my first few weeks I performed the initial triage of a security incident from those logs. I ran a Kusto Query Language (KQL) query in Sentinel to surface failed sign-on events against a host in the environment, and the results showed roughly 100 failed login attempts. That volume pointed to automated password guessing rather than a user mistyping, so I confirmed the activity as a brute-force attempt and identified the source Internet Protocol (IP) address the attempts originated from. My role was pulling the evidence from the sign-on logs, then the analysis and the write-up behind it. I documented the investigation as I went, so my conclusion could be checked by the next person without re-running anything. That investigation shaped how I think about triage. Establish what the activity is, then hand off a finding another person can act on.

Vulnerability scanning and remediation tracking ran through the internship's vulnerability management project. I seeded a Windows Server with weaknesses to stand in for an unmanaged production machine, ran authenticated Tenable Nessus scans against it, and worked the findings in priority order, ranking the fixes by severity and by how quickly each could be done. I cleared the highest-severity items first using PowerShell and Bash, re-scanning after every round to prove each fix and track the totals. Across 6 scan cycles the findings fell from 30 to 3, roughly a 90% reduction, with every critical finding eliminated by the second scan and none returning. I charted the decline across the full cycle so progress was visible at a glance. One finding earned skepticism instead of a fix. A SQLite detection appeared in some scans and vanished from others with no remediation in between, so before touching anything I collected the evidence straight from the host, searching it with PowerShell for SQLite binaries and library files, found none, and documented the detection as a false positive. That cycle of scanning, fixing, and verifying is the clearest way my work has improved a security process, with evidence at every step.

I've also helped maintain secure system configurations by auditing them against a hardening baseline. In a separate internship project I ran a Defense Information Systems Agency Security Technical Implementation Guide (DISA STIG) compliance

audit in Nessus against a Windows 11 machine. The scan checked the machine against 263 controls and flagged 142 as failing. Remediating all 142 would do more harm than good, since over-hardening a machine starts breaking the things it's supposed to do. The real work is selecting the controls that matter for your environment. I chose 10 controls and implemented them with PowerShell, work that included disabling WDigest authentication and turning on PowerShell script block logging so the PowerShell code that runs on the machine gets logged for review. I also removed DES and RC4 from the allowed Kerberos encryption types. After each change I re-ran the audit and didn't close a control until it came back passing.

That configuration work extended to access controls in a personal Amazon Web Services (AWS) cloud lab I built and ran. Secure Shell (SSH) sat exposed on an open inbound port with key files that could leak, so I shut that path down completely and moved host access to AWS Systems Manager (SSM). After the change a session required a temporary credential tied to an authorized identity, and nothing listened on an inbound port anymore. Each connection landed in the CloudTrail log with a record of who opened it and when. My Terraform automation got the same scrutiny once I realized it was running on an administrator identity with keys that never expired. If those keys ever leaked, an attacker would inherit everything that identity could touch. I gave the Terraform automation a role of its own, limited to its automation tasks and nothing else, moved it onto single sign-on so its credentials expired on their own instead of persisting, and retired the old keys.

I also build my own tooling. The best tool I've made to date is a guardrailed automation framework I designed and built in Python around Sentinel, made to complete threat-hunt workflows end to end. It wires a custom Model Context Protocol (MCP) server that executes KQL queries to the hunt platform's application programming interface (API) for question retrieval and answer submission, and a large language model agent does the investigative reasoning inside the harness while the framework enforces the rules. The reliability engineering is mine. Submissions are idempotent and gated, so a low-confidence answer never fires and nothing submits twice. Local state reconciles against the live platform, so a crash or manual edit can't corrupt the source of truth. A postmortem engine mines every wrong submission into a lessons corpus that the workflow surfaces automatically on future questions, converting mistakes into reusable improvements instead of repeated errors. I validated the framework across 10 multi-stage hunts totaling 348 graded questions.

Documentation ties all of this together. The vulnerability and STIG projects above, along with the AWS lab work, are documented publicly on my GitHub, each written up with the steps I took and the evidence that proves the outcome, so a reader who never saw the environment could rebuild the work and check my conclusions. That habit is what useful documentation and procedures are made of. I finished a B.S. in Network Engineering and Security in 2025 and hold the CompTIA Security+, Network+, A+, and Cloud+ certifications. I'd bring that evidence-first approach to protecting the Department of Developmental Services' systems.